

Réseau de Neurones Multicouche sur MNIST

Analyse Comparative : Sigmoid vs ReLU

INF 372 — Sciences des données | Licence 3

YOKO AYMERICK JEFFREY (23U2983)

KAMDOM NGUETCHESSI MERVEILLE PRISCA (23V2060)

KENNE YEMTSA LOWEL RAYANN (23U2477)

Département Informatique

May 2, 2026

Plan de la Présentation

- 1 Contexte et Objectifs du TP
- 2 Architecture du Réseau
- 3 Fonctions d'Activation
- 4 Descente de Gradient Stochastique
- 5 Visualisation des Poids — Heatmaps
- 6 Analyse des Activations de la Couche Cachée
- 7 Comparaison Sigmoid vs ReLU
- 8 Résultats et Évaluation

► Problème posé

Reconnaître des **chiffres manuscrits** (0–9) à partir d'images numériques — classification **multi-classes** supervisée sur le célèbre jeu MNIST.

- ★ Frontières de décision **non linéaires** impossibles à modéliser par régression classique
- Nécessité d'un **réseau de neurones** avec couche cachée
- Accent sur **l'interprétabilité** : visualiser *comment* le réseau apprend

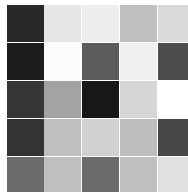


Image 32×32 pixels

→ ?
Classe 0–9

Chaque image : **1024 pixels** aplatis
en vecteur d'entrée normalisé $\in [0, 1]$

Objectifs Précis du TP

▷ Tâches demandées

- 1 Construire un MLP **from scratch** (sans TF/PyTorch dans la version 1)
- 2 Entraîner avec **descente stochastique** (SGD, batch=1)
- 3 Visualiser l'évolution des poids (**heatmap**) jusqu'à convergence
- 4 Analyser les **activations** de la couche cachée par classe

◇ Question centrale

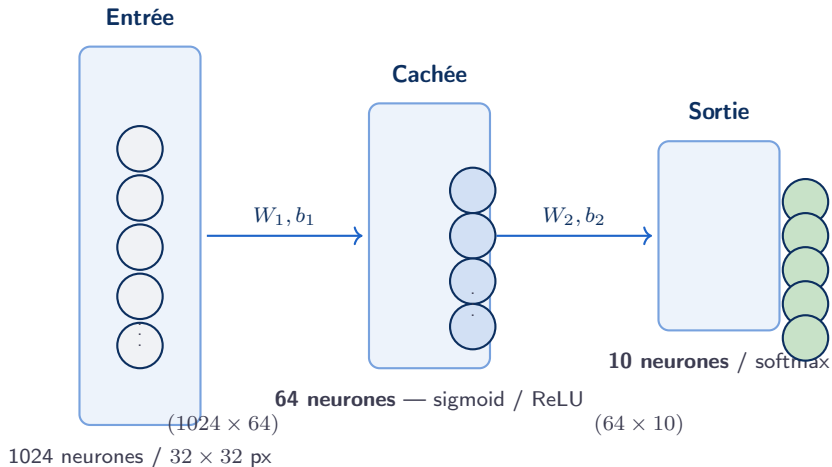
Pourquoi implémenter le même TP avec deux fonctions d'activation différentes ?

→ Comparer empiriquement **Sigmoïde** et **ReLU** sur les mêmes données, avec les mêmes critères d'évaluation.

✓ Apport pédagogique

Comprendre les conséquences *concrètes* du choix d'activation sur la vitesse de convergence, la qualité des représentations et l'interprétabilité.

Architecture du MLP



Paramètres et Hyperparamètres

≡ Architecture commune aux deux versions

Paramètre	Valeur
Entrée	$32 \times 32 = 1024$
Couche cachée	64 neurones
Sortie	10 classes
Activation sortie	Softmax
Perte	Cross-entropie catégorielle
Époques	50

⊗ Initialisation des poids (Xavier)

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{1}{n_{\text{entrée}}}}\right)$$

- Évite la **saturation immédiate** de la sigmoïde
- Maintient la **variance des activations** stable
- Biais initialisés à **zéro**

△ Différences entre les deux TP

	TP1 (Sigmoid)	TP2 (ReLU)
Activation	Sigmoïde	ReLU
lr	0.12	0.01
batch size	1	32

Sigmoïde — Définition et Propriétés

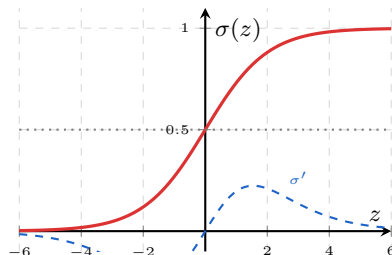
~ Formule

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in]0, 1[$$

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

- Sortie bornée dans $]0, 1[$ — interprétation probabiliste
- $\triangle$ **Problème du gradient qui disparaît** :
pour $|z| \gg 0$, $\sigma'(z) \approx 0$
- La dérivée est calculable directement depuis l'activation : gain de vitesse en backprop
- Nécessite un clipping de z pour stabilité numérique

Courbe Sigmoïde



Zone de saturation = gradient ≈ 0

ReLU — Définition et Propriétés

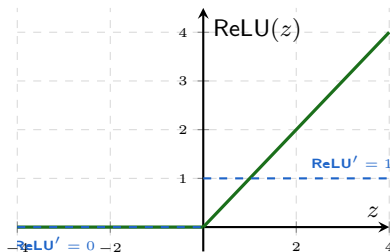
~ Formule

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

- ✓ **Gradient constant** (= 1) sur la partie active : pas de disparition
- ✓ Calcul ultra-rapide : comparaison et seuil
- ✓ Active seulement les neurones utiles (**sparse activations**)
- △ Neurons morts (*dying ReLU*) si $z < 0$ toujours

Courbe ReLU



Gradient non nul sur toute la zone active

La SGD — Propagation Avant et Arrière

→ Forward Pass

$$z_1 = X_b \cdot W_1 + b_1$$

$$a_1 = f(z_1) \quad (\text{sigmoid ou ReLU})$$

$$z_2 = a_1 \cdot W_2 + b_2$$

$$\hat{y} = \text{softmax}(z_2)$$

← Backward Pass

$$\delta_2 = \frac{\hat{y} - y_{\text{onehot}}}{m}$$

$$\nabla W_2 = a_1^\top \cdot \delta_2$$

$$\delta_1 = (\delta_2 \cdot W_2^\top) \odot f'(a_1)$$

$$\nabla W_1 = X_b^\top \cdot \delta_1$$

– Perte : Cross-Entropie

$$\mathcal{L} = -\frac{1}{m} \sum_{i,k} y_{ik} \cdot \log(\hat{y}_{ik} + \varepsilon)$$

↻ Mise à jour

$$W \leftarrow W - \eta \cdot \nabla W, \quad b \leftarrow b - \eta \cdot \nabla b$$

Pourquoi SGD (batch

Bruit dans le gradient → aide à sortir des minima locaux : Convergence irrégulière mais pédagogiquement

Heatmap des Poids — Comment lire ?

Interprétation de la heatmap W_1

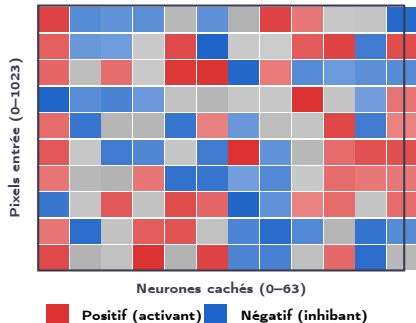
- **Axe Y** : 1024 pixels d'entrée
- **Axe X** : 64 neurones cachés
- **Rouge** : poids positif fort → pixel *active* le neurone
- **Bleu** : poids négatif → pixel *inhibe* le neurone
- **Blanc/neutre** : pixel peu influent

Évolution temporelle

En sauvegardant W_1 à chaque époque, on observe la **spécialisation progressive** des neurones : au fil du temps,

Heatmap W_1 simulée

Époch 1 → Époch 50



Comparaison des Heatmaps de Poids : Sigmoid vs ReLU

≈ Version Sigmoid

- Évolution des poids **très lente** sur les premières époques
- Gradient faible dans les zones saturées → **mise à jour minuscule**
- Aux premières époques, la heatmap reste quasiment **uniforme / blanche**
- La structuration des colonnes n'apparaît qu'après de **nombreuses époques**
- Convergence finale atteinte mais au prix de plus de temps

≈ Version ReLU

- Poids se structurent **dès les premières époques**
- Gradient = 1 sur la partie active → **mises à jour efficaces**
- Colonnes développent des patterns **visuellement distincts** rapidement
- Spécialisation des neurones **visible et prononcée**
- Convergence atteinte en **beaucoup moins d'époques**

⊕ Interprétation clé

Le choix de la fonction d'activation impacte directement la **vitesse à laquelle les poids s'organisent** en

Heatmap des Activations — Quels Neurones s'Activent ?

? Question clé

Pour chaque classe de chiffre (0–9), **quels neurones cachés s'activent préférentiellement** ?

Σ Construction

Pour chaque classe c et neurone i :

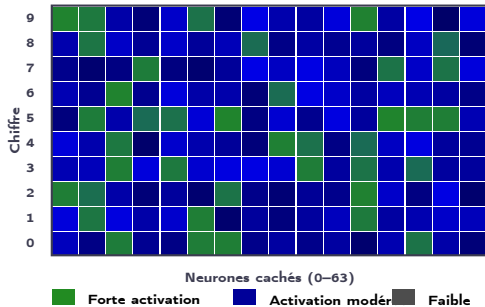
$$\bar{a}_{c,i} = \frac{1}{|S_c|} \sum_{x \in S_c} a_i(x)$$

où S_c = tous les exemples de la classe c .

Résultat : matrice 10×64

Heatmap Activations Moyennes

(matrice 10×64 — simulée)



- **Ligne** = un chiffre (0 à 9)

Neurones « Experts » — Comparaison Sigmoid / ReLU

♣ Sigmoid — Experts peu distincts

- Activations comprises entre 0 et 1, **jamais nulles**
- Chaque neurone réagit à **presque toutes** les classes
- Discrimination inter-classes **floue**
- L'étendue $\max(\bar{a}) - \min(\bar{a})$ par neurone est faible
- Interprétabilité réduite : difficile d'identifier un neurone « spécialiste »

♠ ReLU — Experts bien définis

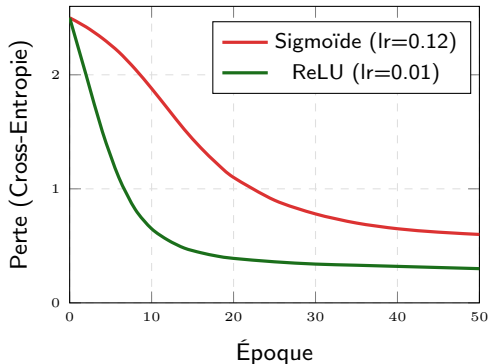
- Activation exactement **nulle** pour beaucoup d'exemples
- Représentations **creuses (sparse)** : chaque neurone réagit à une sous-classe
- Grande étendue d'activation $\Rightarrow$ **neurones discriminants**
- On identifie facilement les **top-5 neurones** par chiffre
- Interprétabilité maximale

= Contribution d'un neurone à la prédiction

$$\text{Contribution}_{i,c} = a_i(x) \times W_{2,i,c}$$

Convergence — Sigmoid vs ReLU

Courbes de perte comparées (simulées)



↗ Observations

- La **sigmoïde** descend lentement : la saturation freine les gradients dès les premières époques
- **ReLU** converge **nettement plus vite** : gradients constants sur la partie active
- Malgré un lr plus faible (0.01 vs 0.12), ReLU atteint une perte inférieure
- La sigmoïde nécessite un **lr plus élevé** pour compenser son gradient faible

✳ Point clé observé

"Sigmoid est vraiment très lent par rapport à"

Tableau Comparatif Complet

Critère	Sigmoïde	ReLU
Plage de sortie	$]0, 1[$ — toujours non nul	$[0, +\infty[$ — exactement nul si $z < 0$
Dérivée	$\sigma(1 - \sigma) \in]0, 0.25]$	0 ou 1
Gradient	S'évanouit pour $ z $ grand	Constant sur zone active
Vitesse de convergence	Lente (besoin de lr élevé)	Rapide
Activations	Denses (jamais nulles)	Creuses (sparse)
Interprétabilité	Moins distincte	Neurones experts clairs
Implémentation	NumPy pur, batch=1	Keras/TF, batch=32
Taux d'apprentissage	0.12	0.01
Stabilité numérique	Clipping nécessaire	Nativement stable
Risque	Vanishing gradient	Dying ReLU

Métriques d'Évaluation

% Précision par classe (exemple type)

Chiffre	Sigmoid	ReLU
0	~92%	~96%
1	~95%	~98%
2	~87%	~93%
3	~88%	~91%
4	~90%	~94%
5	~86%	~90%
6	~91%	~95%
7	~91%	~94%
8	~84%	~89%
9	~88%	~92%
Global	~89%	~93%

† Optimisations appliquées

- **Normalisation** $[0, 1]$: convergence stable
- **Initialisation Xavier** : variance contrôlée
- **Softmax + Cross-entropie** : cohérence classification multi-classes
- **Shuffle** à chaque époque : biais d'ordre évité
- **Mélange stochastique** : exploration des minima

ℓ Limites communes

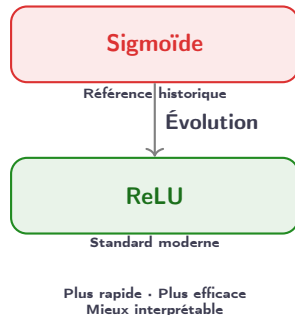
- Architecture à **1 seule couche cachée** (64 neurones)
- Pas de régularisation (dropout, L2)
- Pas d'augmentation de données

Pourquoi avoir utilisé deux fonctions d'activation ?

• Motivation pédagogique principale

L'objectif n'était pas simplement d'obtenir la meilleure précision, mais de **comprendre empiriquement** les effets du choix d'activation :

- 1 Observer le **vanishing gradient** de la sigmoïde en action
- 2 Mesurer la **différence de vitesse** sur les mêmes données
- 3 Comparer la **qualité des représentations** (heatmaps, experts)
- 4 Illustrer pourquoi l'industrie a **abandonné la sigmoïde** en couches cachées



Conclusion Générale

✓✓ Ce que nous avons accompli

- Construit un MLP complet (from scratch & Keras) sur MNIST 32×32
- Implémenté et compris la **SGD** (batch=1 et batch=32)
- Visualisé l'évolution des poids via **heatmaps**
- Analysé les **neurones experts** par classe
- Mené une comparaison rigoureuse **Sigmoid vs ReLU**

★ Conclusion principale

ReLU domine clairement la sigmoïde en couche cachée :

- Convergence **3 à 5× plus rapide**
- Précision finale **supérieure**
- Représentations plus **interprétables**
- Gradients **non dégénérés**

↑ Perspectives

Batch Normalization · Dropout · Adam · Architectures profondes (CNN) pour MNIST

Merci de votre attention — Questions ?